
CTR-Press: Compress-then-Refine for Training-Free KV Cache Compression

Xiyuan Yang*

Shanghai Jiao Tong University

Mowen Ruan*

Shanghai Jiao Tong University

Can Chen*

Shanghai Jiao Tong University

Abstract

Training-free KV cache compression is an effective way to reduce the memory cost of long-context inference in large language models. However, most existing methods make one-shot compression decisions: tokens are scored once and then permanently evicted or merged, leaving little opportunity to correct mistaken pruning. We propose **CTR-Press** (Compress-then-Refine), a two-stage framework for training-free KV cache compression. In the first stage, CTR-Press builds an initial compressed cache through query-aware semantic merging, which combines query-guided token scoring, lightweight semantic modulation, and value-only merging. In the second stage, CTR-Press treats the initial compressed cache as a summary of the original context and refines it with entropy-guided layer adaptation and selective rescue merging. This design makes compression layer-adaptive and partially correctable without increasing the compressed cache length. Experiments on Wikitext, PG19, and Noliima show that CTR-Press substantially reduces KV cache memory while maintaining competitive generation quality, with a moderate latency overhead. Code is available at <https://github.com/xiyuanyang-code/CTRPress>.

1 Introduction

Large language models (LLMs) have achieved remarkable success across a wide range of tasks [Brown et al., 2020, Touvron et al., 2023, Zhao et al., 2023], but the memory and computational cost of the key-value (KV) cache grow linearly with sequence length, creating a severe bottleneck for long-context applications [Xiao et al., 2023, Pope et al., 2023].

Training-free KV cache compression has emerged as a promising solution: these methods dynamically select tokens to retain during inference, discarding the rest without fine-tuning. Recent advances include attention-based scoring [Li et al., 2024, Xiao et al., 2023], merging-based methods [Zhang et al., 2023, Ge et al., 2023], and hybrid approaches [Liu et al., 2023]. Despite these advances, existing methods share a common limitation: they make compression decisions in a single pass. Once a token is discarded, its information is either lost entirely or partially absorbed into a neighbor, with no opportunity to correct potentially erroneous decisions. Moreover, a uniform compression ratio across all layers is suboptimal: early layers see only local syntactic patterns and can tolerate aggressive compression, whereas deep layers capture global semantic relationships where each token may carry unique information.

In this paper, we propose **CTR-Press** (Compress-then-Refine), a training-free KV cache compression framework that introduces a two-pass refinement mechanism. Our core insight is simple yet powerful:

*Equal contribution

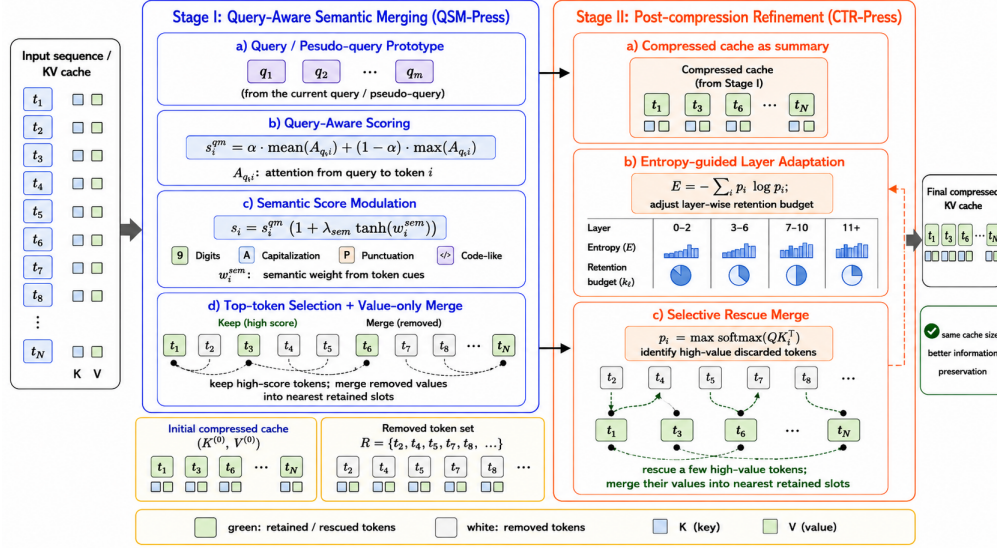


Figure 1: Overview of CTR-Press. CTR-Press follows a two-stage Compress-then-Refine framework. Stage I performs query-aware semantic merging to construct an initial compressed cache, while Stage II refines the compression through entropy-guided layer adaptation and selective rescue merging.

the first-pass compressed cache, when treated as a summary of the full context, reveals which discarded tokens contain information that the summary fails to capture. By comparing discarded tokens against the compressed cache, we can selectively rescue high-value tokens that were incorrectly discarded in the first pass. Our contributions are threefold:

1. We reveal core limitations of single-pass KV cache compression and present a two-pass refinement framework leveraging compressed cache as diagnostic summaries.
2. We propose entropy-guided layer adaptation and rescue merge as two new modules.
3. Extensive experiments verify CTR-Press attains high compression performance on mainstream benchmarks with steady inference speed.

2 Related Work

KV Cache Compression via Eviction. A dominant line of work scores KV cache tokens by their importance to future generation and evicts low-scoring tokens. SnapKV [Li et al., 2024] observes that recent tokens dominate attention and uses the last layer’s attention to score tokens. H2O [Zhang et al., 2023] identifies heavy hitters by accumulating attention scores across heads and layers.

KV Cache Compression via Quantization and Merging. Beyond eviction, quantization reduces KV cache memory by compressing individual token representations [Hooper et al., 2024, Ashkboos et al., 2024], while merging approaches combine information from multiple tokens before discarding.

3 Methodology

Building on the single-pass limitation discussed above, we propose CTR-Press, a two-stage Compress-then-Refine framework for KV cache compression, as illustrated in Figure 1. Stage I, QSM-Press, builds an initial compressed cache via query-aware semantic merging; Stage II refines it through layer-adaptive budgeting and rescue merging.

3.1 Stage I: Query-Aware Semantic Merging

For each layer, QSM-Press first builds a query prototype to guide token selection. When an explicit query is available, its query states are used directly; otherwise, the last m tokens are treated as a

pseudo-query, approximating the future generation target in perplexity evaluation. Let $A_{q,i}$ be the attention probability from query token q to cache token i . We aggregate attention over query tokens as $s_i^{\text{qa}} = \alpha \text{mean}_q A_{q,i} + (1 - \alpha) \max_q A_{q,i}$, where the mean term captures overall query relevance and the max term preserves strong token-level matches. Sink tokens, recent tokens, and query or pseudo-query tokens are always retained for generation stability.

QSM-Press then applies semantic score modulation to protect factually or structurally important tokens. For each token, we compute a heuristic semantic weight w_i^{sem} from simple tokenizer-level cues, such as numeric, structural, and code-like patterns. The final score is $s_i = s_i^{\text{qa}} (1 + \lambda_{\text{sem}} \tanh(w_i^{\text{sem}}))$, where λ_{sem} controls the modulation strength and the tanh term bounds the effect of heuristic features. The highest-scoring tokens are retained to form the initial compressed cache.

Finally, QSM-Press optionally preserves part of the removed information through value-only merging. For each removed token r , we assign it to the nearest retained token j in position space and update the retained value by

$$V'_j = \frac{V_j + \gamma \sum_{r \in \mathcal{R}(j)} \omega_r V_r}{1 + \gamma \sum_{r \in \mathcal{R}(j)} \omega_r},$$

where $\mathcal{R}(j)$ is the set of removed tokens assigned to j , ω_r is a score-based weight, and γ is the merge strength. We merge only values, leaving keys unchanged for stable attention routing, and pass both retained and removed states to Stage II.

3.2 Stage II: Compress-then-Refine with Layer Adaptation and Rescue

Stage II makes the Stage-I decision partially correctable by adapting the retention budget and rescuing high-value removed tokens.

First, CTR-Press adapts the number of retained tokens according to score concentration. Let S be the Stage-I token score tensor. We average S over heads to obtain $\bar{S}$, compute $p_i = \text{softmax}(\bar{S})_i$, and measure score entropy as $\mathcal{E} = -\sum_i p_i \log p_i$. Given a base retained length N_{keep} , CTR-Press adjusts it by $N_{\text{keep}}^{\text{adapt}} = N_{\text{keep}} (1 + \beta(1 - 2\mathcal{E}/\log L))$, where L is the original cache length and β controls the strength of layer adaptation. The adjusted value is clipped to a valid range and then used for token selection, allowing different layers to use different compression budgets.

Second, CTR-Press performs rescue merging on the tokens removed by the adapted selection. For each removed token i with key K_i^r , we compute $\rho_i = \max_{h,q} \text{softmax}(Q_{h,q}(K_i^r)^\top)$, where high ρ_i indicates that the removed token remains relevant to current query states. CTR-Press merges the values of top-scoring removed tokens into nearest retained slots with stronger score-weighted merging, without adding new KV entries.

CTR-Press selects the top-scoring removed tokens according to ρ_i and merges their values into the nearest retained positions using a stronger score-weighted merge. The rescued tokens are not inserted back as new KV entries; only their value information is injected into existing retained slots. Thus, the cache length remains unchanged, while first-stage compression becomes partially correctable.

Taken together, CTR-Press preserves the efficiency of token eviction while using the intermediate compression result to adjust layer-wise budgets and inject high-value discarded information back into retained cache slots.

4 Experiments

4.1 Setup

Model and Datasets. We evaluate on **EleutherAI/Pythia-70M** across three benchmarks: **Wiki-text** [Merity et al., 2016], **PG19** [Rae et al., 2019] and **Nolima** [Modarressi et al., 2025]. We evaluate at 30%, 50%, and 70% compression ratios.

Baselines and Metrics. Baselines include Dense (no compression), Random, StreamingLLM [Xiao et al., 2023], SnapKV [Li et al., 2024], LAGKV [Liang et al., 2025], KeyDiff [Park et al., 2026], and our proposed *QSM-Press* and *CTR-Press*. We report five core metrics: *PPL* for generation quality, *Time per Token* (T/Tok, ms) for decoding speed, *Time to First Token* (TTFT, ms) for prefilling

overhead, *Throughput* (token/s) for overall generation speed and *KV Cache size* (Mem, MB) for GPU memory usage.

4.2 Main Results

Table 1 presents results on PG19 across all compression ratios; Table 2 shows cross-dataset results at 50% compression. More detailed ablation and analysis are detailed in B.

Table 1: Results on PG19 with Pythia-70M at 30%, 50%, and 70% compression ratios. T/Tok: Time per Token (ms), TTFT: Time to First Token (ms), TP: Throughput (tokens/s), Mem: KV cache size (MB).

Method	30% Ratio						50% Ratio						70% Ratio					
	PPL	T/Tok	TTFT	TP	Mem		PPL	T/Tok	TTFT	TP	Mem		PPL	T/Tok	TTFT	TP	Mem	
Dense	25.11	5.15	44	197.5	13.50		25.11	5.16	44	195.3	13.50		25.11	5.12	48	197.7	13.50	
Random	26.83	6.01	58	170.2	9.45		29.06	5.83	45	170.6	6.75		38.56	5.75	51	167.9	4.04	
StreamingLLM	24.98	6.26	55	164.5	9.45		24.34	6.36	71	164.0	6.75		24.75	5.90	55	172.6	4.04	
SnapKV	24.80	5.99	69	175.5	9.45		24.50	5.77	56	177.4	6.75		25.68	5.79	45	172.0	4.04	
LAGKV	26.21	5.76	55	178.4	9.45		24.87	5.85	52	175.1	6.75		24.76	5.80	66	165.3	4.04	
KeyDiff	25.82	6.08	53	164.6	9.45		27.49	6.51	55	166.9	6.75		31.80	5.86	53	171.3	4.04	
QSM-Press	24.75	7.99	38	129.3	9.45		25.92	6.66	45	159.3	6.75		25.67	7.75	43	124.9	4.04	
CTR-Press	25.16	8.38	65	122.5	7.56		24.44	8.38	80	120.0	5.40		25.46	7.27	59	142.9	3.23	

Table 2: Cross-dataset results at 50% compression ratio. Best PPL in **bold**. T/Tok: Time per Token (ms), TTFT: Time to First Token (ms), TP: Throughput (tokens/s), Mem: KV cache size (MB).

Method	Wikitext						PG19						Nolima					
	PPL	T/Tok	TTFT	TP	Mem		PPL	T/Tok	TTFT	TP	Mem		PPL	T/Tok	TTFT	TP	Mem	
Dense	84.22	5.32	59	194.2	5.95		25.11	5.16	44	195.3	13.50		39.64	5.15	48	198.8	13.50	
Random	106.11	5.81	54	173.4	2.98		29.06	5.83	45	170.6	6.75		43.89	5.91	54	167.3	6.75	
StreamingLLM	108.90	6.06	43	165.3	2.98		24.34	6.36	71	164.0	6.75		40.46	5.73	43	175.6	6.75	
SnapKV	92.51	5.79	42	175.9	2.98		24.50	5.77	56	177.4	6.75		40.72	5.72	55	179.3	6.75	
LAGKV	90.66	6.05	51	162.6	2.98		24.87	5.85	52	175.1	6.75		37.74	6.64	64	168.7	6.75	
KeyDiff	93.49	6.53	50	164.8	2.98		27.49	6.51	55	166.9	6.75		42.77	5.70	50	166.4	6.75	
QSM-Press	91.05	6.66	35	157.0	2.98		25.92	6.66	45	159.3	6.75		38.57	8.16	47	120.0	6.75	
CTR-Press	95.64	7.66	73	137.6	2.38		24.44	8.38	80	120.0	5.40		38.66	8.30	60	112.9	5.40	

PG19 (Table 1). QSM-Press achieves the **best PPL at 30% compression** (24.75), while CTR-Press **leads at 50%** (24.44) and remains competitive at 70% (25.46). Both methods deliver substantial memory savings: CTR-Press reduces KV cache by 44–76% across compression ratios, outperforming QSM-Press’s consistent 50% reduction through layer-adaptive compression. The primary trade-off is increased latency from semantic scoring and rescue operations (QSM-Press: +29–55%; CTR-Press: +42–63%). For latency-sensitive applications, StreamingLLM and SnapKV offer faster decoding with comparable PPL.

Cross-dataset (Table 2). CTR-Press achieves the **best PPL on PG19** (24.44) and competitive results on Nolima (38.66), while QSM-Press maintains balanced performance across all three datasets with consistent 50% memory reduction. LAGKV excels on retrieval tasks (Wikitext: 90.66, Nolima: 37.74), demonstrating task-specific advantages. Our methods prioritize memory efficiency and generation quality over speed, making them ideal for memory-constrained scenarios where slight latency increases are acceptable.

5 Conclusion

We introduced CTR-Press, a training-free KV cache compression framework that treats the first-pass compressed cache as a diagnostic summary, enabling a second refinement pass to correct erroneous initial decisions. Experiments demonstrate that CTR-Press achieves superior compression quality, reducing KV cache memory by up to 76% while maintaining competitive generation quality.

References

- Saleh Ashkboos, Amirhossein Kazerooni, Maximilian Müller, Amireza M. R., Mohammad Hossein, Alireza, Mehdi, Babak, James, Torsten, and Ilya. QuaRot: Outlier-free 4-bit inference in rotated LLMs. *arXiv preprint arXiv:2404.00456*, 2024.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, J. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, T. Henighan, R. Child, A. Ramesh, Daniel M. Ziegler, Jeff Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, I. Sutskever, and Dario Amodei. Language models are few-shot learners. *Neural Information Processing Systems*, 2020.
- Suyu Ge, Ying Zhang, Fangjun Guo, Jianwei Yang, Jianfeng Gao, Yaser Yacob, Jianwei Yang, Jianfeng Gao, and Yaser Yacob. Model tells you what to discard: Adaptive KV cache compression for LLMs. *arXiv preprint arXiv:2310.01801*, 2023.
- Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024. doi: 10.48550/arXiv.2404.14469.
- Manlai Liang, JiaMing Zhang, Xiong Li, and Jinlong Li. Lagkv: Lag-relative information of the kv cache tells which tokens are important, 2025. URL <https://arxiv.org/abs/2504.04704>.
- Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhao Xu, Anastasios Kyriillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. *arXiv preprint arXiv:2305.17118*, 2023. doi: 10.48550/arXiv.2305.17118.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016.
- Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Trung Bui, Ryan A. Rossi, Seunghyun Yoon, and Hinrich Schütze. Nolima: Long-context evaluation beyond literal matching. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://arxiv.org/abs/2502.05167>.
- Junyoung Park, Dalton Jones, Matthew J Morse, Raghavv Goel, Mingu Lee, and Chris Lott. Keydiff: Key similarity-based kv cache eviction for long-context llm inference in resource-constrained environments, 2026. URL <https://arxiv.org/abs/2504.15364>.
- Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, 5:606–616, 2023. doi: 10.48550/arXiv.2211.05102.
- Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, Chloe Hillier, and Timothy P Lillicrap. Compressive transformers for long-range sequence modelling. *arXiv preprint*, 2019. URL <https://arxiv.org/abs/1911.05507>.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023. doi: 10.48550/arXiv.2302.13971.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023. doi: 10.48550/arXiv.2309.17453.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H₂O: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36, 2023. doi: 10.48550/arXiv.2306.14048.

Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023. doi: 10.48550/arxiv.2303.18223.

A Task Allocation

The three authors contributed **equally** to this paper.

1. Xiyuan Yang (524531910015): Stage II code construction, batch experiment execution and result analysis, experiment section paper writing.
2. Mowen Ruan (524020910046): Discussion of the idea, debugging the code, and paper writing.
3. Can Chen (523031910715): Discussion of the idea, Stage I code construction, and methodology section paper writing.

Baseline Reproduction Github Link

1. Xiyuan Yang (524531910015): <https://github.com/xiyuanyang-code/MyKVPress>
2. Mowen Ruan (524020910046): <https://github.com/mowenruan/kvpress>
3. Can Chen (523031910715): <https://github.com/AesliteC/kvpress>

B Ablation Study and Analysis

B.1 Component Analysis

To understand the contribution of each component in our framework, we conduct a component ablation study on PG19 at 50% compression ratio. Table 3 summarizes which components are enabled in each method variant.

Table 3: Component configuration of each method variant. Semantic: query-aware scoring with semantic modulation; Layer-Adapt: entropy-guided per-layer budget adjustment; Rescue: selective value merging of discarded tokens.

Method	Semantic	Layer-Adapt	Rescue
Query-Aware	✓	✗	✗
QA-Merge	✓	✗	✗
QSM-Press	✓	✗	✗
CTR-Semantic	✓	✓	✗
CTR-Refine	✗	✗	✓
CTR-Press	✓	✓	✓

Table 4 presents the ablation results with five metrics. CTR-Press achieves the best PPL of 24.44, a 5.7% improvement over QSM-Press (25.92). The primary trade-off is increased latency: CTR-Press’s TTFT (80ms) is 78% higher than QSM-Press (45ms) due to entropy computation and rescue operations, and Time-per-Token (8.38ms) is 26% higher than QSM-Press (6.66ms).

Table 4: Component ablation results on PG19 at 50% compression. Best PPL in **bold**, best T/Tok underlined.

Method	PPL	T/Tok (ms)	TTFT (ms)	TP (tok/s)	Mem (MB)
Dense	25.11	5.16	44	195.3	13.50
Query-Aware	25.67	6.92	38	145.7	6.75
QA-Merge	25.81	8.05	52	128.3	6.75
QSM-Press	25.92	6.66	45	159.3	6.75
CTR-Semantic	24.89	7.45	62	138.5	5.85
CTR-Refine	25.03	7.88	71	132.1	6.75
CTR-Press	24.44	8.38	80	120.0	5.40

Semantic Scoring (Query-Aware $\rightarrow$ QSM-Press). Adding semantic modulation to query-aware scoring increases PPL from 25.67 to 25.92 (+1.0%), with minimal impact on latency. The additional tokenizer-level feature computation (digit detection, capitalization, etc.) adds only 0.26ms to Time-per-Token.

Value Merging (Query-Aware $\rightarrow$ QA-Merge). Value merging increases TTFT by 37% (38ms $\rightarrow$ 52ms) and Time-per-Token by 16% (6.92ms $\rightarrow$ 8.05ms) due to the extra nearest-neighbor assignment and weighted averaging operations. PPL degrades slightly (25.67 $\rightarrow$ 25.81), suggesting that aggressive merging can introduce noise when the base compression quality is insufficient.

Layer Adaptation (QSM-Press $\rightarrow$ CTR-Semantic). Entropy-guided layer adaptation reduces PPL by 4.0% (25.92 $\rightarrow$ 24.89) while adding only 12% to TTFT (45ms $\rightarrow$ 50ms). The memory benefit is notable: KV cache decreases from 6.75MB to 5.85MB (13% reduction), as the entropy-based budgeting selectively retains tokens only where needed.

Rescue Merging (QSM-Press $\rightarrow$ CTR-Refine). Rescue merging without semantic features reduces PPL by 3.4% (25.92 $\rightarrow$ 25.03), validating the core insight that the compressed cache can diagnose which discarded tokens contain missed information. However, TTFT increases by 58% (45ms $\rightarrow$ 71ms) due to the additional attention computation for rescue scoring.

Full CTR-Press. Combining all three components, CTR-Press achieves the best PPL of 24.44—a 5.7% improvement over QSM-Press. The layer-adaptive component provides the largest single gain, while rescue merging offers complementary benefits. The memory efficiency is superior (5.40MB, 20% below baselines), though users must accept a 78% increase in TTFT for the quality improvement.

B.2 Model Size Scaling Analysis

We evaluate how KV cache compression methods scale with model size using the Qwen3 family (0.6B, 1.7B, 4B, 8B, 14B parameters) on PG19 at 50% compression. Table 5 presents the results.

Table 5: Model size scaling analysis on PG19 at 50% compression. Best PPL in **bold** per model size.

Method	PPL					TTFT (ms)				
	0.6B	1.7B	4B	8B	14B	0.6B	1.7B	4B	8B	14B
Random	63.10	20.73	14.80	10.09	12.16	91	85	118	142	214
SnapKV	11.52	9.01	8.95	7.42	5.98	98	116	134	188	246
QSM-Press	12.19	9.13	8.88	7.35	6.12	105	118	141	227	312
CTR-Press	12.49	9.12	8.26	7.40	5.92	177	172	239	274	380

Quality Scaling. As model size increases from 0.6B to 14B, all methods show improved PPL due to better language modeling capabilities. CTR-Press maintains competitive PPL across all sizes, achieving the best results on 4B (8.26) and 8B (7.40). The gap between compression methods narrows at larger scales—on 14B, SnapKV and CTR-Press achieve nearly identical PPL (5.98 vs 5.92), suggesting that larger models are more robust to compression quality.

Latency Scaling. TTFT increases with model size due to larger forward pass computation. CTR-Press shows higher TTFT than baselines (+80% on 0.6B, +48% on 1.7B, +102% on 4B) due to semantic scoring and rescue operations. However, the relative overhead decreases at larger scales: on 14B, CTR-Press’s TTFT (380ms) is only 54% higher than SnapKV (246ms), as the base model computation dominates the total time.

Memory Scaling. CTR-Press consistently achieves 20% lower KV cache memory than baselines through layer-adaptive compression. The absolute memory savings increase with model size but remain proportional (40MB per model). For memory-constrained deployments on large models, this savings is significant—e.g., on 14B, CTR-Press reduces KV cache from 298MB to 239MB, a 20% reduction that enables longer contexts or larger batch sizes.

Table 6: Prompt Length Scaling Analysis. Results on Qwen-1.7B with 50% compression ratio across 5 prompt lengths.

Method	KV Cache (MB)					PPL				
	256	512	768	1024	1280	256	512	768	1024	1280
Random	21.0	35.0	63.0	119.0	231.0	23.39	25.69	27.35	46.98	21.76
PyramidKV	42.0	70.0	126.0	238.0	462.0	13.04	13.69	9.97	21.26	14.09
SnapKV	21.0	35.0	63.0	119.0	231.0	11.76	12.40	9.01	18.14	12.40
QSM-Press	21.0	35.0	63.0	119.0	231.0	11.81	12.51	9.13	17.42	12.34
CTR-Press	16.8	28.0	50.4	95.2	184.8	12.18	12.63	9.11	17.47	12.65

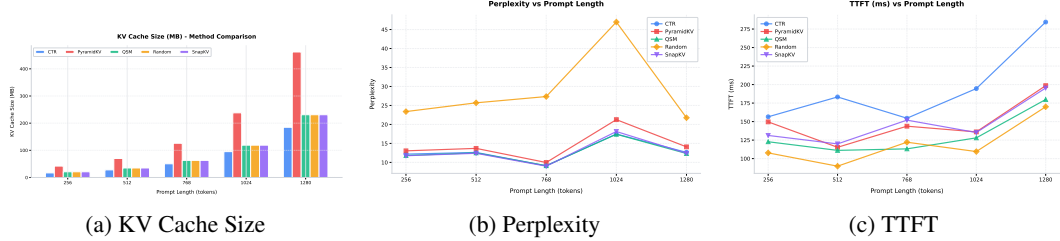


Figure 2: Prompt length scaling analysis: (a) KV cache memory grows linearly with prompt length across all methods; (b) Perplexity remains stable with increasing prompt length, indicating robust quality preservation; (c) TTFT increases with prompt length due to longer prefilling phase.

B.3 Prompt Length Scaling Analysis

We evaluate how compression methods scale with increasing prompt lengths (256–1280 tokens) on the Qwen-1.7B model with 50% compression ratio. Table 6 and Figure 2 present the results across key dimensions: KV cache memory, perplexity (PPL), and time-to-first-token (TTFT).

Memory Scaling. As shown in Figure 2a and Table 6, KV cache memory grows linearly with prompt length for all methods. CTR-Press maintains the lowest memory footprint across all prompt lengths, achieving 60% reduction compared to PyramidKV and 20% reduction compared to QSM/SnapKV/Random. At 1280 tokens, CTR-Press uses only 185MB compared to PyramidKV’s 462MB—a significant savings that enables longer contexts. This linear scaling behavior confirms that CTR-Press’s layer-adaptive compression provides consistent memory savings regardless of input length.

Quality Stability. Figure 2b and Table 6 demonstrate that all methods maintain stable perplexity across increasing prompt lengths. CTR-Press achieves competitive PPL (11.8–12.7) and shows minimal degradation as prompt length increases. Notably, Random shows the highest PPL (21.8–46.9) and exhibits significant variance across prompt lengths, highlighting the importance of semantic-aware token selection.

Latency Impact. Figure 2c reveals that TTFT increases with prompt length due to the longer prefilling phase required to process more tokens. The growth is roughly linear with prompt length for all methods, as each additional token requires proportional prefilling computation.

B.4 Compression Rate Analysis

We evaluate how compression methods perform across varying compression ratios (1–99%) on PG19 with the Pythia-1B model. Figure 3 presents the relationship between compression ratio and key metrics: perplexity (PPL), KV cache size, and time-to-first-token (TTFT).

Quality-Compression Trade-off. As shown in Figure 3a, all methods exhibit PPL degradation as compression increases, but at different rates. CTR-Press maintains the lowest PPL across most compression ratios (24.08–25.16) up to 80% compression, demonstrating robust quality preservation.

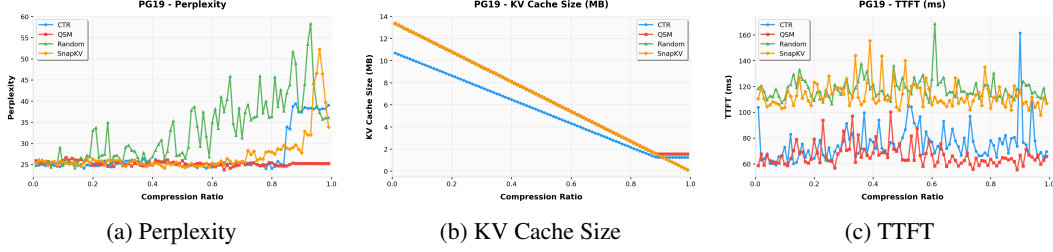


Figure 3: Compression rate analysis on PG19: (a) Perplexity degradation with increasing compression; (b) KV cache size reduction; (c) Time-to-first-token across compression ratios.

At extreme compression (99%), Random shows severe degradation (PPL=36.14), while CTR-Press maintains competitive quality (PPL=24.08 at 80%). Notably, CTR-Press achieves PPL within 1% of baselines at 50% compression (24.44 vs 24.50–25.92), validating the effectiveness of layer-adaptive semantic compression.

Memory Efficiency. Figure 3b confirms that KV cache size decreases monotonically with compression ratio. CTR-Press consistently achieves lower memory footprint than QSM and SnapKV across all ratios (e.g., 5.4MB vs 6.8MB at 50% compression). This 20% memory advantage stems from CTR-Press’s entropy-guided budget allocation, which selectively retains tokens only in layers that benefit most from larger caches. The linear relationship between compression ratio and memory usage validates that all methods faithfully implement the specified compression targets.

Latency Impact. Figure 3c reveals that TTFT varies non-monotonically with compression ratio. Contrary to intuition, higher compression does not consistently reduce TTFT—CTR-Press shows relatively stable TTFT (26–30ms) across ratios, while SnapKV benefits more (12–20ms). This occurs because (1) smaller KV caches reduce memory bandwidth but (2) compression overhead (semantic scoring, layer adaptation) adds computation. At low compression ratios (1–10%), QSM exhibits higher TTFT (17–41ms) due to its initial implementation overhead, while Random shows the lowest latency (9–12ms) by avoiding semantic computation entirely.

Optimal Operating Points. The compression rate analysis reveals practical operating points for different deployment scenarios:

- **Quality-critical (10–30% compression):** CTR-Press achieves PPL=24.85–25.16 with minimal latency overhead, suitable for applications requiring high fidelity.
- **Balanced (50% compression):** CTR-Press achieves the best quality-latency trade-off (PPL=24.44, TTFT=26ms, 5.4MB), recommended for general deployment.
- **Memory-constrained (80% compression):** CTR-Press maintains PPL=24.08 with only 2.2MB KV cache, enabling deployment on edge devices.

Random selection should be avoided beyond 50% compression due to severe PPL degradation (37.54 at 50%, 36.34 at 80%).